

I. STATE-SPACE MODELLING

One of the most effective methods of modelling systems is through the State-Space method. It combines the existing state matrix, $x(t)$, and combines it with the input, $u(t)$, to determine the changes made on the state, $\dot{x}(t)$, and the output of the system, $y(t)$. The equation describing the relation is as follows:

$$\dot{x}(t) = \mathbf{A}x(t) + \mathbf{B}u(t) \quad (1)$$

$$y(t) = \mathbf{C}x(t) + \mathbf{D}u(t) \quad (2)$$

Where $\mathbf{A}$, $\mathbf{B}$, $\mathbf{C}$, and $\mathbf{D}$ are matrices of real numbers, with dimensions informed by those of the state, input, and output matrices.

A. Applications of the State-Space

State-Space matrices are useful for condensing equations of motion into a simpler matrix relating input to output coordinates. They offer some of the best time domain representations of systems, providing combined information density with physical intuition that is better than that of a transfer function representation. Besides this, state-space matrices are required for the dynamics inversion approach planned to be utilized the body angle controller.

B. Producing the State-Space Model

Recall the equations of motion for the system produced earlier:

$$\left(m_b + m_w + \frac{I_w}{r_w^2}\right) \ddot{x} + m_b \ell \cos \theta \ddot{\theta} - m_b \ell \sin \theta \dot{\theta}^2 = -\frac{M_{\text{mot}}}{r_w}. \quad (3)$$

And

$$(I_b + m_b \ell^2) \ddot{\theta} + m_b \ell \cos \theta \ddot{x} - m_b g \ell \sin \theta = M_{\text{mot}}. \quad (4)$$

Remember that the state-space model seeks to represent how the system's state adjusts relative to the inputs existing state, and how the system's output responds to changes in the state. In linear form, we specifically care about behavior very near to a specific point. In the case of our system, we care about the output θ near the state $[x, \dot{x}, \theta, \dot{\theta}] = [0, 0, 0, 0]$. Knowing this gives us enough information to produce our linearized state-space model.

Firstly, we must linearize the equations of motion around the operating point $[x, \dot{x}, \theta, \dot{\theta}] = [0, 0, 0, 0]$. At this point, we note our approximations:

$$\cos \theta \approx 1 \quad (5)$$

$$\sin \theta \approx 0 \quad (6)$$

$$\dot{\theta}^2 \approx 0 \quad (7)$$

Applying these approximations to the equations of motion yields the linearized equations:

$$\left(m_b + m_w + \frac{I_w}{r_w^2}\right) \ddot{x} + m_b \ell \ddot{\theta} = -\frac{M_{\text{mot}}}{r_w}. \quad (8)$$

And

$$(I_b + m_b \ell^2) \ddot{\theta} + m_b \ell \ddot{x} - m_b g \ell \theta = M_{\text{mot}}. \quad (9)$$

Multiplying (9) by r_w and adding the two equations together yields:

$$\left(r_w \left(m_b + m_w + \frac{I_w}{r_w^2}\right) + m_b \ell\right) \ddot{x} + (r_w m_b \ell + m_b \ell^2 + I_b) \ddot{\theta} = m_b g \ell \theta \quad (10)$$

We begin deriving the state-space model by setting up the state vector and input vector for the linearized system. In our case, the state vector is

$$\mathbf{x} = \begin{bmatrix} x \\ \dot{x} \\ \theta \\ \dot{\theta} \end{bmatrix} \quad (11)$$

and the input vector is

$$\mathbf{u} = [M_{\text{mot}}]. \quad (12)$$

By this, we get:

$$\dot{\mathbf{x}} = \begin{bmatrix} \dot{x} \\ \ddot{x} \\ \dot{\theta} \\ \ddot{\theta} \end{bmatrix} \quad (13)$$

Setting up the state-space and consulting the equations allows for us to derive the values for the $\mathbf{A}$ and $\mathbf{B}$. We consider it to be a trivial exercise for the interested reader to consider why $\mathbf{C} = \begin{bmatrix} 0 & 0 & 1 & 0 \end{bmatrix}$ and $\mathbf{D} = \begin{bmatrix} 0 \end{bmatrix}$. The matrix representation of the equations of motion is useful to evaluate relationships:

The matrices are with constants k_n and provide expressions for their numeric values:

$$\mathbf{A} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & k_1 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & k_2 & 0 & 0 \end{bmatrix}, \quad \mathbf{B} = \begin{bmatrix} 0 \\ k_3 \\ 0 \\ k_4 \end{bmatrix} \quad (14)$$

Where:

$$k_1 = \quad (15)$$

$$k_2 = \quad (16)$$

$$k_3 = \quad (17)$$

$$k_4 = \quad (18)$$

II. DERIVING DYNAMICS INVERSION

Dynamics Inversion is a controls approach that relies on physics modeling to predict optimal inputs for a desired effect. We utilize the state space matrix to work backwards and acquire the desired behaviors. As a note, this approach relies on inverting non-square or non-invertible matrices, and as such, we must use approximations. The derivation begins with the standard state-space equation, except replacing $y(t)$ with some arbitrary desired output v :

$$\dot{x}(t) = \mathbf{A}x(t) + \mathbf{B}u(t) \quad (19)$$

$$v = \mathbf{C}x(t) + \mathbf{D}u(t) \quad (20)$$

Since our system will have a matrix $\mathbf{D}$ with completely 0 components (meaning there is lag between input and change), we must find another way to relate the input to the change in output. This is done by differentiating the output equation:

$$\dot{v} = \mathbf{C}\dot{x}(t) \quad (21)$$

$$\dot{v} = \mathbf{C}(\mathbf{A}x(t) + \mathbf{B}u(t)) \quad (22)$$

We don't have a desired change in output, so we introduce an arbitrary time constant τ for a discrete derivative:

$$\dot{v} = \frac{v - y(t)}{\tau} \quad (23)$$

$$\frac{v - y(t)}{\tau} = \mathbf{C}(\mathbf{A}x(t) + \mathbf{B}u(t)) \quad (24)$$

Now that the equation is fully derived, we can extract a formula for $u(t)$ in terms of the other variables:

$$\mathbf{C}^{-1} \frac{v - y(t)}{\tau} = \mathbf{A}x(t) + \mathbf{B}u(t) \quad (25)$$

$$\mathbf{C}^{-1} \frac{v - y(t)}{\tau} - \mathbf{A}x(t) = \mathbf{B}u(t) \quad (26)$$

$$\mathbf{B}^{-1} \mathbf{C}^{-1} \frac{v - y(t)}{\tau} - \mathbf{A}x(t) = u(t) \quad (27)$$

$$u(t) = \frac{\mathbf{B}^{-1} \mathbf{C}^{-1} (v - y(t)) - \mathbf{A} \mathbf{B}^{-1} \tau x(t)}{\tau} \quad (28)$$

A. Addressing Matrix Pseudo-Inversions

For dynamics inversion, we have been required to invert matrices that are not intended to be inverted. As a result, we require a method of matrix pseudo-inversion. This will be done through the Moore-Penrose pseudo-inverse. The Moore-Penrose pseudo-inverse of the matrix $\mathbf{A}$, $\mathbf{A}^\dagger$, obeys the following rules:

$$\mathbf{A} \mathbf{A}^\dagger \mathbf{A} = \mathbf{A} \quad (29)$$

$$\mathbf{A}^\dagger \mathbf{A} \mathbf{A}^\dagger = \mathbf{A}^\dagger \quad (30)$$

$$(\mathbf{A} \mathbf{A}^\dagger)^T = \mathbf{A} \mathbf{A}^\dagger \quad (31)$$

$$(\mathbf{A}^\dagger \mathbf{A})^T = \mathbf{A}^\dagger \mathbf{A} \quad (32)$$

We can calculate this matrix using the singular value decomposition of the matrix $\mathbf{A}$.

$$\mathbf{A} = \mathbf{U} \mathbf{\Sigma} \mathbf{V}^T \quad (33)$$

Where $\mathbf{U}$ and $\mathbf{V}$ are orthogonal and unitary matrices representing rotation, and $\mathbf{\Sigma}$ is a diagonal matrix representing scaling;

$$\mathbf{A}^\dagger = \mathbf{V} \mathbf{\Sigma}^\dagger \mathbf{U}^T \quad (34)$$

Where $\mathbf{\Sigma}^\dagger$ is made up of the reciprocals of the elements of $\mathbf{\Sigma}$

Given this, we may update our dynamics inversion equation:

$$u(t) = \frac{\mathbf{B}^\dagger \mathbf{C}^\dagger (v - y(t)) - \mathbf{A} \mathbf{B}^\dagger \tau x(t)}{\tau} \quad (35)$$